DATABASE MANAGEMENT SYSTEMS

CT 652 | BE / BCT | Year III / Part II

Tribhuvan University — Institute of Engineering

CHAPTER 9: ADVANCED DATABASE CONCEPTS

Question Bank with Comprehensive Answers — OODBMS, ORDBMS, XML, Distributed Databases, Data Warehousing, OLAP, Data Mining, Association Rules

SYLLABUS OVERVIEW — Unit 9

Unit	Topic	Hours	Marks
1	Introduction	3 hrs	4
2	Data Models	7 hrs	12
3	Relational Languages & Model	7 hrs	12
4	Constraints & Normalization	6 hrs	12
5	Query Processing & Optimization	4 hrs	8
6	File Structure & Hashing	4 hrs	8
7	Transactions & Concurrency Control	6 hrs	12
8	Crash Recovery	4 hrs	6
9	Advanced Database Concepts	4 hrs	6
Total		**45 hrs**	**80**

Chapter 9 Topics (4 Hours | 6 Marks)

- 9.1 Object-Oriented Databases (OODBMS)
- 9.2 Object-Relational Databases (ORDBMS)
- 9.3 XML and Databases
- 9.4 Distributed Databases — fragmentation, replication, transparency
- 9.5 Data Warehousing — ETL, star schema, snowflake schema
- 9.6 OLAP Operations — roll-up, drill-down, slice, dice, pivot
- 9.7 Data Mining — classification, clustering, association rules

CHAPTER 9 — Questions & Comprehensive Answers

Q: What is an Object-Oriented Database (OODBMS)? Explain its key features: OID, encapsulation, inheritance, polymorphism. Compare OODBMS with RDBMS.
Asked: 2081 Chaitra; 2079 Ashwin; 2078 Poush; 2075 Bhadra; 2074; 2072 Ashwin; 2071 Bhadra; 2070 Magh — half of all exams | Marks: [4]

Answer

Object-Oriented Database (OODBMS)

An Object-Oriented Database Management System (OODBMS) is a database system that stores data as objects — combining the capabilities of a database (persistence, concurrency, querying, recovery) with those of an object-oriented programming language (classes, objects, inheritance, encapsulation). It was developed to handle complex data types that relational databases handle poorly: multimedia, CAD/CAM, scientific data, and software engineering data.

Key Features of OODBMS

1. Object Identity (OID)

Every object in an OODBMS is assigned a unique, system-generated Object Identifier (OID) that is immutable — it never changes even if the object's attribute values change. The OID is independent of the object's content (unlike a relational primary key, which is based on data values). Objects can reference each other via OIDs.

```
Relational DB: Employee identified by EmpID=E001 (value-based — changes if
value changes)
OODBMS:        Employee object has OID=0x7FA23 (system-generated, permanent,
content-free)

OID advantages:
  • Identity is independent of attribute values
  • Supports object sharing and circular references
  • No need for foreign keys — direct object references via OID
```

2. Encapsulation (Data Abstraction)

An object bundles together its data (attributes) and the operations (methods) that act on that data into a single unit — the class. The internal state of an object is hidden from the outside world; it can only be accessed and modified through the object's defined methods (interface). This is the principle of information hiding.

```
class Employee {
  private: int empID;  string name;  double salary;  // hidden data
  public:
    double getSalary() { return salary; }   // accessor method
    void setSalary(double s) {              // mutator method
      if (s > 0) salary = s;               // validation inside method
    }
    double computeBonus() { return salary * 0.10; }
```

```
};

Benefit: Internal representation can change without affecting external users.
```

3. Inheritance

A class (subclass / derived class) can inherit attributes and methods from another class (superclass / base class). The subclass extends or overrides the inherited behaviour. This supports code reuse and models IS-A relationships naturally.

```
class Person {                    // Superclass
  string name; string address;
  void display();
};

class Employee extends Person {   // Subclass — inherits name, address,
display()
  int empID; double salary;
  void computeBonus();            // new method
};

class Manager extends Employee {  // Multi-level inheritance
  int numReports;
  void assignTask();
};

Types: Single, Multiple, Multi-level, Hierarchical inheritance
```

4. Polymorphism

The same method name can have different implementations for different classes. At runtime, the correct version is selected based on the actual type of the object (dynamic / late binding). This allows a single interface to represent different underlying forms.

```
class Shape { virtual double area() = 0; };
class Circle  extends Shape { double area() { return pi*r*r; } };
class Rectangle extends Shape { double area() { return w*h; } };

Shape* s;
s = new Circle(5);     s->area();  // calls Circle::area()  = 78.5
s = new Rectangle(4,6); s->area(); // calls Rectangle::area() = 24

One method name 'area()' — multiple implementations depending on object type.
```

5. Complex Objects and Type Constructors

OODBMS supports complex, nested object structures using type constructors: set, list, bag (multiset), array, and tuple. An attribute can itself be an object or a collection of objects — not just atomic values as in RDBMS.

```
class Department {
  string dname;
  set<Employee> employees;   // attribute is a collection of Employee objects
  Employee manager;          // attribute is an object reference
};
```

OODBMS vs RDBMS — Comparison

Aspect	RDBMS	OODBMS
Data model	Tables (relations) of rows and columns	Objects with attributes and methods
Data types	Atomic types only (int, string, date)	Complex types: objects, sets, lists, arrays
Identity	Based on primary key (data values)	System-generated OID (immutable, content-free)
Relationships	Foreign keys + JOINs	Direct object references via OID
Encapsulation	None — data and logic separate	Yes — data and methods bundled in class
Inheritance	Not supported	Supported — class hierarchies
Polymorphism	Not supported	Supported — method overriding
Querying	SQL — powerful, standardized	OQL (Object Query Language) — less standardized
Complex data	Difficult — must flatten to tables	Natural — objects can be nested
Performance	Excellent for simple data, JOINs	Better for complex navigational queries
Maturity / Adoption	Very mature; universal adoption	Less mature; niche adoption (CAD, multimedia)
Examples	Oracle, MySQL, PostgreSQL, SQL Server	db4o, ObjectDB, Versant, GemStone

Q: What is an Object-Relational Database (ORDBMS)? How does it extend the relational model? Explain collection types, row types, and user-defined types.
Asked: 2080 Ashwin; 2079 Jestha; 2077 Chaitra; 2073 Magh; 2070 Bhadra | Marks: [4]

Answer

Object-Relational Database (ORDBMS)

An Object-Relational Database Management System (ORDBMS) extends the traditional relational model by adding object-oriented features — while still keeping the relational foundation (tables, SQL, ACID transactions). It provides the best of both worlds: the power and familiarity of SQL with the expressiveness of object-oriented concepts.

SQL:1999 (SQL3) standard introduced these extensions. PostgreSQL is the most prominent ORDBMS.

Extensions to the Relational Model

1. User-Defined Types (UDTs)

Users can define their own data types, going beyond built-in SQL types. A UDT can have attributes and methods, similar to a class in OOP.

```
-- Define a user-defined type 'Address'
CREATE TYPE Address AS (
  street   VARCHAR(50),
  city     VARCHAR(30),
  zip_code CHAR(5)
);

-- Use UDT as a column type
CREATE TABLE Employee (
  empID   INT PRIMARY KEY,
  name    VARCHAR(50),
  home_addr  Address,    -- UDT as column type
  work_addr  Address     -- multiple instances of same UDT
);

-- Access UDT attributes:
SELECT e.home_addr.city FROM Employee e WHERE empID = 101;
```

2. Collection Types (Nested Tables and Arrays)

ORDBMS allows an attribute to hold a collection of values, breaking the 1NF restriction of plain relational databases.

```
-- Array type: fixed or variable length array
CREATE TYPE PhoneArray AS VARCHAR(15) ARRAY[3];

CREATE TABLE Employee (
  empID   INT,
  name    VARCHAR(50),
  phones  PhoneArray    -- array of phone numbers
);

-- Nested table (multiset / bag)
CREATE TYPE SkillSet AS TABLE OF VARCHAR(30);

CREATE TABLE Employee (
  empID   INT,
  skills  SkillSet      -- nested table of skills
);
```

3. Row Types (Structured Types as Rows)

Tables can be typed — each row is an instance of a named structured type, with attributes and methods.

```
CREATE TYPE PersonType AS (
  name    VARCHAR(50),
  dob     DATE
) NOT FINAL;   -- allows subtypes

CREATE TYPE EmployeeType UNDER PersonType AS (
  empID   INT,
  salary  DECIMAL(10,2)
);

CREATE TABLE Employees OF EmployeeType;  -- typed table
```

4. Inheritance (Table Inheritance)

ORDBMS supports type hierarchies and table inheritance. A subtable inherits all attributes and can add more. Queries on a parent table can also return rows from subtables.

```
-- PostgreSQL example:
CREATE TABLE Person (name TEXT, dob DATE);
CREATE TABLE Employee () INHERITS (Person);  -- inherits name, dob
  -- Employee also has empID, salary added separately

SELECT * FROM Person;   -- returns rows from Person AND Employee
SELECT * FROM ONLY Person;  -- returns ONLY Person rows (not Employee)
```

5. User-Defined Functions (UDFs)

Users can define custom functions and procedures in SQL or external languages (C, Java, Python) and use them in SQL queries.

```
CREATE FUNCTION distance(p1 Point, p2 Point) RETURNS FLOAT
LANGUAGE SQL AS $$
  SELECT SQRT((p2.x - p1.x)^2 + (p2.y - p1.y)^2);
$$;

SELECT name, distance(location, POINT(0,0)) AS dist_from_origin
FROM Offices
ORDER BY dist_from_origin;
```

Feature	Pure RDBMS	ORDBMS	Pure OODBMS
Data types	Atomic only	Atomic + complex UDTs	Full object types
Collections	Not allowed (1NF)	Arrays, nested tables	Sets, lists, bags
Inheritance	Not supported	Type and table inheritance	Full class hierarchy
Query language	SQL	Extended SQL (SQL:1999)	OQL or proprietary
OID	No (uses key values)	Optional (REF type)	Yes — system OIDs
Maturity	Highest	High (PostgreSQL, Oracle)	Lower

Q: What is XML? How is XML used in databases? Explain DTD, XML Schema, XPath, and XQuery.

Asked: 2078 Chaitra; 2075 Baisakh; 2071 Magh | Marks: [4]

Answer

XML (Extensible Markup Language)

XML (Extensible Markup Language) is a self-describing, text-based format for representing structured and semi-structured data. Unlike HTML (fixed tags for presentation), XML allows users to define their own tags for meaning. XML is widely used for data exchange between heterogeneous systems — different DBMS, different operating systems, web services.

XML Document Example

```
<?xml version="1.0" encoding="UTF-8"?>
<employees>
  <employee empID="E001">
    <name>Ram Sharma</name>
    <department>Engineering</department>
    <salary>75000</salary>
    <skills>
      <skill>Java</skill>
      <skill>SQL</skill>
    </skills>
  </employee>
  <employee empID="E002">
    <name>Sita Thapa</name>
    <department>Marketing</department>
    <salary>65000</salary>
  </employee>
</employees>
```

DTD (Document Type Definition)

A DTD defines the structure (schema) of an XML document — which elements can appear, in what order, and with what attributes. DTDs are written in a non-XML syntax.

```
<!DOCTYPE employees [
  <!ELEMENT employees (employee+)>
  <!ELEMENT employee (name, department, salary, skills?)>
  <!ATTLIST employee empID ID #REQUIRED>
  <!ELEMENT name (#PCDATA)>
  <!ELEMENT department (#PCDATA)>
  <!ELEMENT salary (#PCDATA)>
  <!ELEMENT skills (skill+)>
  <!ELEMENT skill (#PCDATA)>
]>

Symbols: + = one or more; * = zero or more; ? = optional
#PCDATA = parsed character data (text content)
```

XML Schema (XSD)

XML Schema (XSD — XML Schema Definition) is a more powerful alternative to DTD. It is itself written in XML, supports data types, namespaces, and more complex structural constraints.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="employee">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name" type="xs:string"/>
        <xs:element name="salary" type="xs:decimal"/>
      </xs:sequence>
      <xs:attribute name="empID" type="xs:ID" use="required"/>
    </xs:complexType>
  </xs:element>
</xs:schema>

Advantages over DTD: typed data, namespace support, written in XML itself
```

XPath (XML Path Language)

XPath is a query language for selecting nodes from an XML document using path expressions. It navigates the tree structure of XML using a path syntax similar to a file system.

```
/employees/employee          → all <employee> elements
/employees/employee/@empID   → all empID attributes of employees
//skill                      → all <skill> elements anywhere in document
/employees/employee[salary > 70000]/name  → names of highly paid employees
/employees/employee[1]       → first employee element
```

XQuery

XQuery is a full query language for XML data, analogous to SQL for relational data. It uses the FLWOR expression (For-Let-Where-Order by-Return).

```
for $emp in /employees/employee
where $emp/salary > 70000
order by $emp/name
return
  <result>
    <name>{ $emp/name/text() }</name>
    <dept>{ $emp/department/text() }</dept>
  </result>

This is equivalent to SQL:
  SELECT name, department FROM employee WHERE salary > 70000 ORDER BY name
```

XML in Databases

- Native XML databases: store and query XML directly (eXist-db, BaseX, MarkLogic)
- XML-enabled RDBMS: store XML in special XML columns, query with XPath/XQuery (Oracle XMLType, PostgreSQL xml type, SQL Server xml)
- Data exchange: XML is the standard format for web services (SOAP), configuration files, data import/export between different DBMS
- Semi-structured data: XML handles irregular, optional, and nested structures that relational tables cannot represent naturally

---

Q: What is a distributed database? Explain its features, advantages, disadvantages, fragmentation types, replication, and transparency.

Asked: 2082 Kartik; 2081 Ashwin; 2079 Chaitra; 2078 Baishakh; 2076; 2075 Baisakh; 2073; 2072 Magh; 2071 Magh — majority of all exams | Marks: [4]

Answer

Distributed Database

A distributed database is a collection of logically related data that is physically stored across multiple computers (nodes / sites) connected by a computer network. To the user, a distributed database appears as a single unified database — the distribution is transparent. Each site has its own local DBMS, and the distributed DBMS (DDBMS) coordinates operations across all sites.

```
Site A (Kathmandu)          Site B (Pokhara)          Site C (Biratnagar)
┌──────────────────┐        ┌──────────────────┐       ┌──────────────────┐
│ Local DB: Nepal  │        │ Local DB: Gandaki│       │ Local DB: Koshi  │
│ Branch data      │        │ Branch data      │       │ Branch data      │
│ Local DBMS       │        │ Local DBMS       │       │ Local DBMS       │
└──────────┬───────┘        └──────────┬───────┘       └──────────┬───────┘
           │                           │                          │
           └───────────────────────────┴──────────────────────────┘
                                Network (LAN/WAN/Internet)
                                          │
                              Distributed DBMS Layer
                        (makes distribution transparent)
```

Features of Distributed Databases

- Physical distribution: Data is stored at multiple geographically separated sites
- Logical integration: Despite physical separation, users see one unified database
- Autonomous local DBs: Each site can operate independently; has its own local transactions
- Network communication: Sites communicate via a network to exchange data and coordinate transactions
- Heterogeneity (optional): Different sites may run different DBMS software (heterogeneous DDBMS)

Transparency Features

Transparency means the user is not aware of or affected by the distribution. The DDBMS provides several levels of transparency:

Transparency Type	Description
Distribution transparency	User does not know which data is at which site
Fragmentation transparency	User does not know data is fragmented across sites
Replication transparency	User does not know copies exist at multiple sites
Location transparency	Queries work regardless of data location
Concurrency transparency	Concurrent access handled invisibly
Failure transparency	System continues working even if some sites fail

Advantages of Distributed Databases

- Improved performance: Local queries are processed at the local site — no network overhead for local data
- Higher availability and reliability: If one site fails, others continue functioning — no single point of failure
- Scalability: New sites can be added to the network to increase capacity
- Data sharing: Users at different locations can share the same logically unified database
- Economics: Multiple cheaper computers can outperform one large expensive mainframe
- Local autonomy: Each site has control over its own data while sharing the overall system

Disadvantages of Distributed Databases

- Complexity: Much more complex than centralized DBMS — distributed query processing, transaction management, concurrency control across sites
- Security: More exposure points — data transmitted over network can be intercepted
- Network overhead: Distributed queries require data to be transmitted over the network — expensive for large joins
- Difficult to maintain consistency: Keeping replicated data consistent across sites is challenging
- High cost: Network infrastructure, distributed DBMS software, skilled administrators
- Deadlock detection: More complex — must detect deadlocks across multiple sites

Fragmentation

Fragmentation is the process of dividing a relation into smaller pieces (fragments) that are stored at different sites. Each fragment contains a subset of the original relation's data.

1. Horizontal Fragmentation

The relation is divided by rows — each fragment contains a subset of tuples satisfying a certain predicate. Different rows go to different sites.

```
Employee(EmpID, Name, Dept, Salary, Branch)

H1: σ Branch='Kathmandu' (Employee) → stored at Site A
H2: σ Branch='Pokhara'   (Employee) → stored at Site B
H3: σ Branch='Biratnagar'(Employee) → stored at Site C

Reconstruction: Employee = H1 ∪ H2 ∪ H3  (UNION of fragments)

Best for: Queries that access data from one region only — no network overhead
```

2. Vertical Fragmentation

The relation is divided by columns — each fragment contains a subset of attributes. The primary key must appear in each fragment to allow reconstruction.

```
Employee(EmpID, Name, Dept, Salary, Address, Phone)

V1: π EmpID, Name, Dept (Employee)          → HR site
V2: π EmpID, Salary (Employee)              → Payroll site
V3: π EmpID, Address, Phone (Employee)      → Admin site

Reconstruction: Employee = V1 ⋈ V2 ⋈ V3  (NATURAL JOIN on EmpID)

Best for: Queries that access only a subset of columns
```

3. Mixed (Hybrid) Fragmentation

A combination of horizontal and vertical fragmentation. A relation is first horizontally fragmented, then each horizontal fragment is further vertically fragmented — or vice versa.

```
First horizontal: H1 = σ Branch='KTM' (Employee)
Then vertical:    H1V1 = π EmpID, Name (H1)
                  H1V2 = π EmpID, Salary (H1)
```

Replication

Replication means maintaining copies (replicas) of the same data at multiple sites. If one site fails, the data is still available at other sites.

- Full replication: Entire database is copied at every site — maximum availability, very expensive to update (must update all copies)
- Partial replication: Some fragments are replicated at some sites — balance between availability and update cost
- No replication: Each fragment stored at exactly one site — simple updates but lower availability

---

Q: What is data warehousing? Explain its characteristics, ETL process, architecture, star schema, and snowflake schema. Differentiate data warehouse and data mart.

Asked: 2082; 2081 Ashwin; 2079 Chaitra; 2078 Chaitra; 2077; 2075; 2074; 2073 Magh; 2072; 2071; 2070 — almost all exams | Marks: [4]

Answer

Data Warehouse

A data warehouse is a large, subject-oriented, integrated, time-variant, non-volatile collection of data used to support management decision making. It is a central repository of historical data consolidated from multiple operational source systems (OLTP systems, flat files, external sources). Data in a warehouse is read-only — no operational updates occur.

Characteristics (W. H. Inmon's 4 Properties)

1. Subject-Oriented

Data is organized around major business subjects (customers, products, sales, employees) rather than around business processes (order entry, invoicing). This makes the data easier to analyze.

2. Integrated

Data from multiple heterogeneous sources (different databases, formats, naming conventions) is cleaned, transformed, and integrated into a consistent format before loading into the warehouse.

3. Time-Variant

Data in a warehouse covers a long time horizon (typically 5–10 years), unlike operational databases that hold current data only. Each record has a time dimension — historical trends can be analyzed.

4. Non-Volatile

Once data is loaded into the warehouse, it is not modified. Only new data is added periodically (nightly, weekly). No UPDATE or DELETE on existing records — only INSERT and SELECT.

ETL Process (Extract, Transform, Load)

ETL is the pipeline by which data flows from operational source systems into the data warehouse:

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────────────┐
│  Source      │    │   EXTRACT    │    │  TRANSFORM   │    │      LOAD        │
│  Systems     │──▶ │  Pull data   │──▶ │  Clean,      │──▶ │  Insert into     │
│  (OLTP DBs,  │    │  from all    │    │  integrate,  │    │  Data Warehouse  │
│  flat files, │    │  sources     │    │  aggregate   │    │  (staging area   │
│  external)   │    │              │    │              │    │  → warehouse)    │
└──────────────┘    └──────────────┘    └──────────────┘    └──────────────────┘
```

- Extract: Read data from various source systems (OLTP databases, spreadsheets, web logs, external APIs)
- Transform: Clean data (remove duplicates, fix errors), standardize formats (date formats, currency), resolve naming conflicts, compute derived attributes, aggregate data to the required granularity
- Load: Insert the transformed data into the data warehouse tables in bulk (typically overnight/weekly batch process)

Data Warehouse Architecture

```
Source Systems → ETL Process → Data Warehouse → OLAP / Reporting Tools → End
                                                                  Users

                                Detailed architecture:
┌──────────┐    ┌──────────┐    ┌──────────────────────────┐    ┌──────────┐
│ OLTP DBs │    │Staging   │    │   Data Warehouse         │    │ OLAP     │
│ ERP, CRM │──▶ │Area      │──▶ │  (Fact + Dim tables)     │──▶ │ Cube     │
│ Flat Files│   │(ETL work)│    │  Data Marts (subsets)    │    │ Reports  │
└──────────┘    └──────────┘    └──────────────────────────┘    │Dashboards│
                                                                 └──────────┘
```

Star Schema

The star schema is the simplest and most common data warehouse schema. It has a central fact table surrounded by dimension tables — resembling a star shape.

```
Fact Table: SALES_FACT
  (time_key FK, product_key FK, store_key FK, customer_key FK,
   units_sold, revenue, cost)   ← measures (numeric, aggregatable)

Dimension Tables (denormalized):
  TIME_DIM    (time_key PK, date, month, quarter, year)
  PRODUCT_DIM (product_key PK, name, category, brand, supplier)
  STORE_DIM   (store_key PK, store_name, city, region, country)
  CUSTOMER_DIM(customer_key PK, name, age, gender, city)

               TIME_DIM
                  |
  PRODUCT_DIM - SALES_FACT - STORE_DIM
                  |
             CUSTOMER_DIM

Advantages: Simple queries (few JOINs), fast query performance
Disadvantages: Dimension tables may have redundancy (denormalized)
```

Snowflake Schema

The snowflake schema is a normalized version of the star schema. Dimension tables are further normalized into multiple related tables — the diagram looks like a snowflake.

```
PRODUCT_DIM is split into:
  PRODUCT_DIM (product_key PK, name, category_key FK)
  CATEGORY_DIM (category_key PK, category_name, brand_key FK)
  BRAND_DIM    (brand_key PK, brand_name, supplier_key FK)
  SUPPLIER_DIM (supplier_key PK, supplier_name, country)

Advantages: Reduces redundancy; saves storage space
Disadvantages: More complex queries; more JOINs; slower performance
```

Data Warehouse vs Data Mart

Aspect	Data Warehouse	Data Mart
Scope	Enterprise-wide — all business subjects	Department-specific — e.g., Sales, Finance, HR
Size	Very large (TBs to PBs)	Smaller (GBs to tens of TBs)
Data source	Multiple operational systems across the enterprise	Usually subset of data warehouse or single source
Time to build	Months to years	Weeks to months
Users	Entire organization	Specific department or business unit
Example	Central bank warehouse with all transactions	Marketing department's customer analytics mart

---

Q: What is OLAP? Explain the OLAP operations: roll-up, drill-down, slice, dice, and pivot with examples. Differentiate OLAP and OLTP.

Asked: 2082 Kartik; 2081 Ashwin; 2078 Baishakh; 2077; 2075 Bhadra; 2073 Magh; 2072 Ashwin; 2071; 2070 — all exams | Marks: [4]

Answer

OLAP (Online Analytical Processing)

OLAP is a category of software technology that enables analysts and managers to examine data from multiple perspectives quickly. OLAP works on a multidimensional data cube — data organized by dimensions (Product, Time, Location) and measures (Sales, Revenue, Profit). OLAP queries are read-only, complex, and involve aggregations over large amounts of historical data.

Multidimensional Data Cube

```
                    3D OLAP Cube — dimensions: Product, Time, Location

                                                Time
                                            (Q1,Q2,Q3,Q4)
                                                  /
                              Location    /————————————/
```

```
                                (KTM,PKR) /──────────────/ │
                                         /──────────────/  │
                                    /  Product    /    │ Revenue
                                   /  (A,B,C,D)  /    │ at each
                                   └──────────────┘     │ cell

            Each cell = Revenue for (Product, Time, Location) combination
                 Aggregating along a dimension gives summary values
```

OLAP Operations

1. Roll-Up (Drill-Up / Aggregation)

Roll-up increases the level of aggregation (moves up the dimension hierarchy). Detail data is summarized to a higher level. For example: from monthly sales to quarterly to annual.

```
Example: Sales data by month × city

  Jan-KTM=100  Jan-PKR=80   Jan-BRT=60
  Feb-KTM=120  Feb-PKR=90   Feb-BRT=70
  Mar-KTM=110  Mar-PKR=85   Mar-BRT=65

Roll-up on Time (month → quarter):
  Q1-KTM = 100+120+110 = 330
  Q1-PKR = 80+90+85 = 255
  Q1-BRT = 60+70+65 = 195

Roll-up on Location (city → region):
  Q1-Region1 = Q1-KTM + Q1-PKR + Q1-BRT = 780
```

2. Drill-Down (Disaggregation)

Drill-down is the reverse of roll-up — it moves from a higher level of aggregation to a lower level of detail. For example: from annual to quarterly to monthly to daily.

```
Example: Annual sales → Quarterly → Monthly → Daily

Start: 2081-Total = 4800
Drill down to quarters:
  Q1-2081=1100  Q2-2081=1200  Q3-2081=1300  Q4-2081=1200
Drill down to months (from Q1):
  Jan=350  Feb=380  Mar=370

Used to: Investigate what is causing a summary trend
```

3. Slice

Slice selects one specific value for one dimension, reducing the 3D cube to a 2D table (a 'slice' of the cube).

```
Example: Slice on Time='Q1'

From 3D cube (Product × Time × Location):
  Fix Time = Q1
  Result: 2D table of Product × Location for Q1 only
```

```
Product     KTM   PKR   BRT
A           120    90    70
B           150   110    80
C           100    80    60

Used to: Analyze performance for a specific time period
```

4. Dice

Dice selects a sub-cube by specifying ranges or multiple values on two or more dimensions simultaneously (unlike slice which fixes exactly one value on one dimension).

```
Example: Dice on Time IN {Q1,Q2} AND Location IN {KTM,PKR}

From 3D cube → extract sub-cube where:
  Time = Q1 or Q2  AND  Location = KTM or PKR

Result: 3D sub-cube (Product × {Q1,Q2} × {KTM,PKR})

Used to: Narrow down analysis to a specific sub-region of data
```

5. Pivot (Rotate)

Pivot rotates the data cube to view it from a different angle — swapping dimensions between rows, columns, and pages. No data changes — only the presentation orientation changes.

```
Before pivot:
         Q1    Q2    Q3    Q4
Product-A 100   120   130   110
Product-B 150   160   155   140

After pivot (swap Time ↔ Product):
         Product-A   Product-B
Q1           100         150
Q2           120         160
Q3           130         155
Q4           110         140

Used to: View data from a different perspective without changing values
```

OLAP vs OLTP

Aspect	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Purpose	Day-to-day operational transactions	Business intelligence, analysis, decision support
Operations	INSERT, UPDATE, DELETE (read + write)	SELECT only — complex aggregation queries
Data type	Current, up-to-date operational data	Historical, integrated data over years
Query type	Simple, predefined (e.g., withdraw Rs.500)	Complex, ad-hoc (e.g., trend over 5 years)

Response time	Milliseconds (fast short transactions)	Seconds to minutes (complex aggregations)
Users	Thousands of concurrent users (clerks, customers)	Hundreds of analysts, managers, executives
Database size	GBs (operational data only)	TBs to PBs (years of history)
Schema	Normalized (3NF) — minimize redundancy	Denormalized (star/snowflake) — maximize speed
Example	ATM withdrawal, order entry, reservation	Monthly sales trend by region and product

---

Q: What is data mining? Explain data mining techniques: classification, clustering, association rule mining. Explain the Apriori algorithm for association rules with support and confidence.

Asked: 2081 Chaitra; 2080 Ashwin; 2079 Ashwin; 2078 Poush; 2076; 2073; 2072 Magh; 2070 Bhadra — majority of all exams | Marks: [4]

Answer

Data Mining

Data mining (also called Knowledge Discovery in Databases, KDD) is the process of automatically discovering hidden patterns, correlations, anomalies, and useful knowledge from large databases. Data mining extracts implicit, previously unknown, and potentially useful information from data — going beyond simple querying to discover patterns the user did not explicitly know to look for.

KDD Process

```
Raw Data → Data Cleaning → Data Integration → Data Selection
    → Data Transformation → DATA MINING → Pattern Evaluation
                                        → Knowledge Presentation
```

Data Mining Techniques

1. Classification

Classification assigns each data instance to one of a predefined set of classes (categories). A classifier is trained on a labelled training dataset and then used to predict the class of new, unseen instances. Classification is supervised learning — the class labels are known during training.

```
Examples:
  • Email spam detection: classify email as SPAM or NOT SPAM
  • Medical diagnosis: classify patient as DIABETIC / NOT DIABETIC based on
lab results
  • Loan approval: classify applicant as APPROVE / REJECT
  • Sentiment analysis: classify review as POSITIVE / NEGATIVE / NEUTRAL

Popular algorithms:
  Decision Tree (ID3, C4.5, CART)
  Naive Bayes Classifier
```

```
  k-Nearest Neighbor (kNN)
  Support Vector Machine (SVM)
  Neural Networks / Deep Learning

Process:
  1. Training phase: build model from labelled training data
  2. Testing phase: evaluate model accuracy on test data
  3. Prediction phase: apply model to new unlabelled data
```

2. Clustering

Clustering groups data instances into clusters such that instances within the same cluster are similar to each other and different from instances in other clusters. Clustering is unsupervised learning — no predefined class labels exist. The algorithm discovers the groups itself.

```
Examples:
  • Customer segmentation: group customers by purchasing behavior
  • Document clustering: group similar news articles
  • Image segmentation: group pixels by color/texture
  • Anomaly detection: points that don't fit any cluster = anomalies

Popular algorithms:
  k-Means Clustering: partition n points into k clusters
    1. Choose k initial centroids randomly
    2. Assign each point to nearest centroid
    3. Recompute centroids as mean of assigned points
    4. Repeat until centroids don't change

  Hierarchical Clustering: build tree of clusters (dendrogram)
  DBSCAN: density-based clustering — finds clusters of arbitrary shape
```

3. Association Rule Mining

Association rule mining finds interesting relationships (co-occurrences) between items in transactional data. The classic example is market basket analysis — finding which products are frequently purchased together.

```
Association rule format:  A => B  [support=s, confidence=c]
  Meaning: If A is bought, B is also likely bought
  Example: {Bread, Butter} => {Milk}  [support=40%, confidence=75%]

Key measures:
  Support(A => B)    = P(A ∪ B) = freq(A and B together) / total transactions
  Confidence(A => B) = P(B|A)   = freq(A and B together) / freq(A)
  Lift(A => B)       = Confidence(A=>B) / Support(B)
    Lift > 1: positive association; Lift = 1: independent; Lift < 1: negative

Example:
  1000 transactions; A+B appear together in 400; A alone in 500; B alone in
600
  Support({A,B}) = 400/1000 = 40%
  Confidence(A=>B) = 400/500 = 80%
  Lift(A=>B) = 0.80 / (600/1000) = 0.80/0.60 = 1.33  → positive association
```

Apriori Algorithm

The Apriori algorithm is the classic algorithm for finding frequent itemsets and generating association rules. It uses the Apriori principle (anti-monotonicity property): if an itemset is infrequent, all its supersets are also infrequent.

```
Apriori Principle: if {A,B} is infrequent, then {A,B,C} is also infrequent.
  → Used for pruning: don't generate supersets of infrequent itemsets.
```

```
Algorithm Apriori(T, min_support, min_confidence):
  // T = transaction database; min_sup = minimum support threshold

  Step 1: Find all frequent 1-itemsets (L1)
    For each item i: count freq(i) in T
    L1 = { i | support(i) >= min_support }

  Step 2: For k = 2, 3, ... until no new frequent itemsets found:
    Ck = candidate k-itemsets (generated from L(k-1))
         By joining L(k-1) with itself
         Prune: remove any k-itemset whose (k-1)-subset is NOT in L(k-1)
    Count support of each candidate in Ck by scanning T
    Lk = { c in Ck | support(c) >= min_support }

  Step 3: Generate association rules from all frequent itemsets
    For each frequent itemset F, for each non-empty subset A of F:
      Rule: A => (F - A)
      If confidence(A => F-A) >= min_confidence: output the rule
```

Worked Example — Apriori

```
Transactions (T):    Items: A, B, C, D, E      min_support=50% (3/5 txns)
  T1: {A, B, C}
  T2: {A, B, D}
  T3: {A, B}
  T4: {B, C, D}
  T5: {A, C}

Pass 1 — Frequent 1-itemsets:
  A: 4 txns ✓   B: 4 txns ✓   C: 3 txns ✓   D: 2 txns ✗   E: 0 txns ✗
  L1 = { {A}, {B}, {C} }  (D and E pruned: below min_support=3)

Pass 2 — Candidate 2-itemsets from L1:
  C2 = { {A,B}, {A,C}, {B,C} }
  Count: {A,B}=3 ✓  {A,C}=3 ✓  {B,C}=2 ✗
  L2 = { {A,B}, {A,C} }

Pass 3 — Candidate 3-itemsets from L2:
  C3 = { {A,B,C} }  (only possible 3-set from {A,B} and {A,C})
  Check: subset {B,C} not in L2 → PRUNE {A,B,C}
  L3 = {} → STOP

Frequent itemsets: {A}, {B}, {C}, {A,B}, {A,C}

Rule generation (min_confidence=60%):
  From {A,B}: A=>B: conf=3/4=75% ✓  B=>A: conf=3/4=75% ✓
  From {A,C}: A=>C: conf=3/4=75% ✓  C=>A: conf=3/3=100% ✓
```

Summary of Data Mining Techniques

Technique	Type	Goal	Output	Example
Classification	Supervised	Predict class label	Classifier model	Spam/Not-Spam
Clustering	Unsupervised	Discover groups	Cluster assignments	Customer segments
Association Rules	Unsupervised	Find co-occurrences	IF-THEN rules	Bread => Milk [60%,80%]
Regression	Supervised	Predict numeric value	Regression model	Predict house price
Anomaly Detection	Unsupervised	Find outliers	List of anomalies	Credit card fraud

SUMMARY TABLE — Chapter 9 Question Coverage

All Chapter 9 questions from TU-IOE BCT examination papers (2070–2082) and their coverage in this document.

No.	Question / Topic	Asked In	Marks
C1	Object-Oriented Database (OODBMS) — features, OID, encapsulation, inheritance, polymorphism	2081 Chaitra, 2079 Ashwin, 2078 Poush, 2075 Bhadra, 2074, 2072 Ashwin, 2071 Bhadra, 2070 Magh	[4]
C2	OODBMS vs RDBMS comparison table	2081 Chaitra, 2079 Ashwin, 2078 Poush, 2075 Bhadra, 2072 Ashwin, 2070 Magh	[4]
C3	Object-Relational Database (ORDBMS) — extending relational model, collection types, row types	2080 Ashwin, 2079 Jestha, 2077 Chaitra, 2073 Magh, 2070 Bhadra	[4]
C4	XML in databases — DTD, XML schema, XPath, XQuery	2078 Chaitra, 2075 Baisakh, 2071 Magh	[4]
C5	Distributed Database — features, advantages, disadvantages, transparency	2082, 2081 Ashwin, 2079 Chaitra, 2078 Baishakh, 2076, 2075 Baisakh, 2073, 2072 Magh, 2071 Magh, 2070 Magh	[4]
C6	Fragmentation — horizontal, vertical, mixed; replication	2081 Ashwin, 2078 Baishakh, 2073, 2072 Magh, 2071 Magh	[4]
C7	Data Warehousing — definition, characteristics, ETL, architecture, star/snowflake schema	2082, 2081 Ashwin, 2079 Chaitra, 2078 Chaitra, 2077, 2075 Bhadra, 2074, 2073 Magh, 2072 Ashwin, 2071, 2070	[4]
C8	OLAP operations — roll-up, drill-down, slice, dice, pivot; OLAP vs OLTP	2082, 2081 Ashwin, 2078 Baishakh, 2077, 2075 Bhadra, 2073 Magh, 2072 Ashwin, 2071, 2070	[4]
C9	Data Mining — definition, techniques: classification, clustering, association rule mining	2081 Chaitra, 2080, 2079 Ashwin, 2078 Poush, 2076, 2073, 2072 Magh, 2070 Bhadra	[4]
C10	Association Rule Mining — support, confidence, lift, Apriori algorithm	2080 Ashwin, 2076 Baisakh, 2072 Magh	[4]

Chapter 9 exam pattern: Q(a) alternates between OODBMS / ORDBMS / XML / Distributed DB; Q(b) alternates between Data Warehousing/OLAP and Data Mining. Prepare all topics. Key formulas: Support(A=>B)=freq(A∪B)/N; Confidence(A=>B)=freq(A∪B)/freq(A); Lift=Confidence/Support(B). OLAP operations: roll-up (aggregate), drill-down (detail), slice (fix 1 dim), dice (range on 2+ dims), pivot (rotate view). Star schema: fact table + denormalized dims. ETL = Extract-Transform-Load.

— End of Chapter 9 Answer Bank —